

EXERCISE 4 - PARTICLE SIMULATION

1 General

1.1 Setup

Exercises 1 to 4 are web-based [THREE.js](#) applications. For developing and running, an editor and a local web server are required. We recommend using [VSCode](#) with the [Live Preview extension](#). After installing the extension, simply open the exercise folder in VSCode, right click `index.html` and click “Show Preview”. If you prefer other editors or web servers, feel free to use them.

If you are unfamiliar with [THREE.js](#), we recommend taking a look the [Fundamentals](#) chapter of their manual. If you want to brush up on modern JavaScript, the [Prerequisites](#) chapter is a good starting point.

1.2 Grading

You can reach a total of 8 points in this exercise. The maximum number of reachable points is indicated at each task. The grading will take place at the exercise presentation session towards the end of the semester (see TUWEL for the date and location). During this session, you will be asked to present a (sub-)task of this exercise to the class. After successfully presenting and explaining the code, you will receive the points. In case you fail to explain the code, you will not receive the points for this exercise sheet.

1.3 Submission Instructions

In the TUWEL submission, upload a ZIP file containing:

- A short video of your program, showcasing the tasks you implemented. This should be no longer than ~ 60 seconds, similar to the video shown along the task description in TUWEL.
- Your source code including the `js` directory with all source files, the `index.html` and `main.css` files.
- A txt file listing all the (sub-)tasks you have implemented and the corresponding points you expect you receive (this will be the basis for your grade in the exercise presentation).

2 Tasks

In this exercise, you will need to make changes within the [WGSL](#) compute shader that updates the particle velocities and positions for each simulation step. All tasks are to be solved within the file `js/particle-update.js`. The corresponding sections in the code are marked at their beginning and end with comments (e.g., `//TASK1 - begin` and `//TASK1 - end`). For this exercise, we are using THREE.js' WebGPU backend. Due to a WebGPU limitation, the framework for this exercise will not work in Firefox. **Therefore, we recommend using Google Chrome.**

2.1 Task 1: Global forces (2 Points)

Add a gravity force. (1 Point)

Add drag force. Its magnitude is $|\text{velocity}|^2 \times \text{DragCoefficient}$ and its direction is opposite to the direction of `velocity`. (1 Point)

2.2 Task 2: Force fields (3 Points)

Implement three types of force fields:

Directional force field: Each particle within the force field should have a force applied to it. (1 Point)

Expansion force field: Each particle within the force field should be pushed away from the center of the force field. (1 Point)

Contraction force field: Each particle within the force field should be pulled towards the center of the force field. (1 Point)

The force fields are stored in an array of structs. Each element has the fields `type`, `position`, `radius` and `force`. The number of active force fields is stored in `numForceFields`. Iterate over all active force fields and sum up their influences. For expansion and contraction force fields, use only the magnitude of `force`, you can ignore its direction.

2.3 Task 3: Collision (3 Points)

Implement basic collisions with box-shaped obstacles. If a collision is detected, you can reflect the velocity vector over the face the particle collided with. Use `bounciness` to determine how large the reflected vector should be. Alternatively, instead of simply reflecting the velocity vector, you can also correctly calculate the exact new position.

The obstacles are stored in an array of structs. Each element has the fields `minCorner` and `maxCorner`. The number of active obstacles is stored in `numActiveObstacles`. Iterate over active obstacles and handle collisions with their faces. Ignore the size of the particles, they may intersect with the boxes.